

CS779 Competition: Machine Translation System for India

Dhruv Gupta

240354

dhruvgupta24@iitk.ac.in

Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This is a report that details the development phase of the Neural Machine Translation (NMT) system for translating Indian languages (Hindi and Bengali), as part of the course project for CS779. A diverse set of architectures were experimented with, starting from basic RNN based sequence models, all the way to an optimized version of the Transformer architecture. The report covers the entire process of developing the model, including challenges faced and things implemented according to research done (with proper citations).

1 Competition Result

Note: for all the below metrics, I'm entering the ranks and scores from the training phase, since I unfortunately couldn't submit anything during the testing phase.

- **Codalab Username:** d_240354
- **Final Leaderboard Rank on the Train Set:** 14th (during TRAINING phase)
- **chrF++ Score corresponding to the Final Rank:** 0.412
- **ROUGE Score corresponding to the Final Rank:** 0.431
- **BLEU Score corresponding to the Final Rank:** 0.157
- **Total Number of Submissions in the Training Phase:** 25
- **Total Number of Submissions in the Testing Phase:** 0

Table showing the Number of Submissions made each Week during the Training Phase:

Week	Submission Count
Week 1	2
Week 2	0
Week 3	2
Week 4	21
Total	25

2 Problem Description

2.1 Some Background About the Current State of MT in India

I would first like to briefly talk about why this is such a relevant problem statement.. Machine Translation for Indian languages is very different from translating English into languages with the same script and structure like French or German because of linguistic diversity and things like dialects

and fundamental structure differences. There are two main problems in training these systems for Indic languages, one is the difficulty in tokenization of Indian languages because of semantic differences (but this is fixed for the most part with sub-word tokenization algorithms like BPE and SentencePiece). The more pressing problem is the availability of datasets for translation, since most Indian languages don't have a lot of digital presence. Some very active efforts like IndicNLP and IndicBERT have significantly improved available datasets and models, but we still have a long way to go and hence I think this was a great project in encouraging hands-on experience in language translation with low resources.

2.2 Problem Statement and Setting

The setting of the project was that we were young engineers in a mass media company that used to primarily publish its material in English but now wants to expand to Indic languages, starting with Hindi and Bengali, so the task was to design a Neural Machine Translation (NMT) system that could translate English text to Hindi and Bengali. Due to resource constraints in this firm, the problem involved using only PyTorch to build an encoder-decoder type architecture from scratch such that the encoder processes the English input sentence to a vector representation and the decoder uses it to generate the Hindi/Bengali translation. The benchmarking and evaluation of these translations was to be done using standard metrics like chrF++, BLEU, and ROUGE scores.

2.3 Rules and Constraints Involved

- All models involved must be trained from scratch in PyTorch (No pre-trained resources from places like HuggingFace and Scikit-Learn are allowed.)
- The only pre-trained resources allowed are static embeddings like Word2Vec or GloVe (although I didn't end up using these).
- Only the datasets provided are allowed to be used.

3 Data Analysis

A structured and comprehensive Exploratory Data Analysis (EDA) session was conducted and it helped to influence the major architectural decisions taken down the line (will explain this further later). It helped tune hyperparameters and also select preprocessing pipelines.

3.1 Corpus Statistics

The number of samples in the train and test data were:

- English to Hindi: 23,085 Test and 80,797 Train samples
- English to Bengali: 19,672 Test and 68,849 Train samples

An important thing to note here is that **2,791 duplicate sentence pairs** were found (across train and test data combined). But, I didn't include a step in pre-processing to remove them since they made up only about 1.8% of the data and hence didn't induce much of a bias in the model.

3.2 Why 25 tokens?

A critical hyperparameter while training the transformer was the maximum sequence length, which has implications on tensor sizes and memory, training speed. So, a good length had to be identified, that maximized efficiency but also retained most information. The plot in figure 1 was generated during EDA, depicting the language-wise sentence length distribution. I inferred from this that **Bengali was the most compact**, with 95.39% of sentences having a length of 25 tokens or fewer, and English and Hindi had sort of similar distributions, with 86.53% and 84.28% of sentences, respectively, falling under the 25 token mark. Hence, the **MAX_LENGTH=25** global was set, since it would allow me to capture the majority of sentences without excess padding being required and compute being wasted. This was motivated by this paper [1] that showed NMT performance can degrade over large sequences.

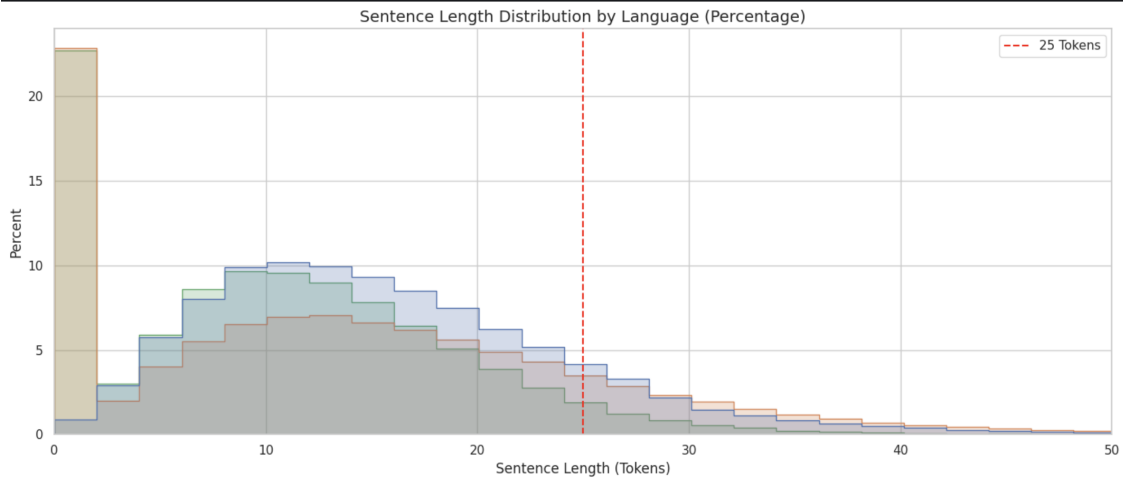


Figure 1: Sentence Length Distribution by Language (Percentage vs Tokens)

3.3 Linguistic Isochrony being Followed?

Continuing from the findings in the previous subsection, I then went on to find out the correlation of sentence lengths between English and the target languages. This is shown in the form of a correlation heatmap in figure 2. Often, in translation from English to for example, German, one language might be really long compared to the other because of things like compound words and underlying grammar. But luckily, the heatmap revealed a pretty close correlation between the languages to English in this case, with a Pearson Coefficient of 0.92 for English-Hindi lengths and 0.88 for English-Bengali lengths. This can be seen in the plot as well because the points are closely clustered around the $y=x$ line. So, this made it clear that I didn't have to worry about changing positional encodings in the transformer model, since the sequence lengths would mostly be the same while translating the sentences. This analysis of source-target length correlation is used commonly for corpus-evaluation and is inspired from the following paper: [2].

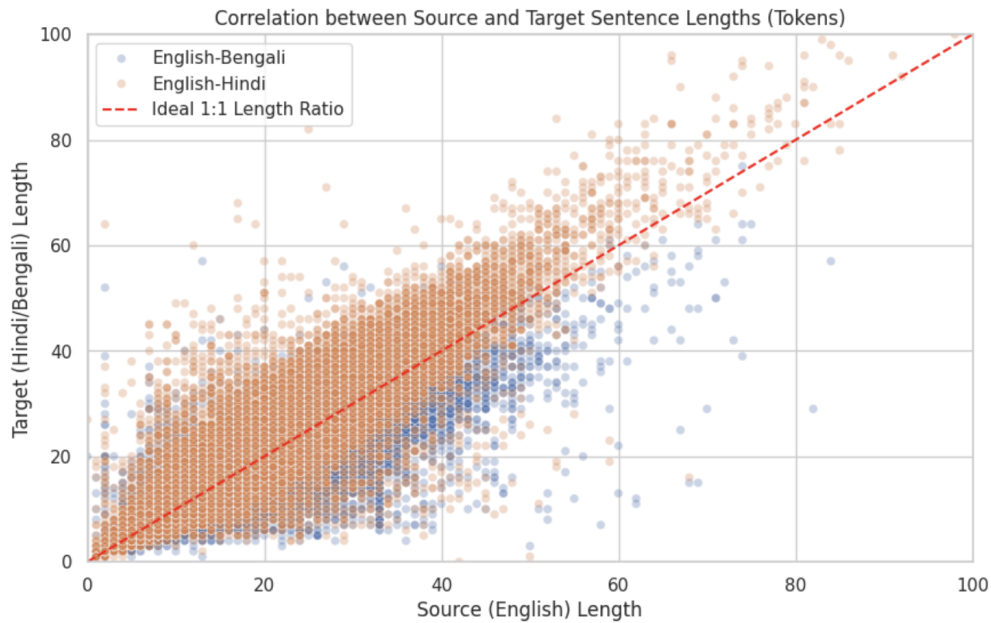


Figure 2: Correlation between Source and Target Lengths

3.4 Vocabulary Distribution Analysis

Lastly, the vocabulary distributions were analysed, ie, the most frequent tokens and if they actually contributed much to the meaning of the sentence. As shown in the histogram in figure 3, the nature of the top 10 tokens revealed the following:

- English: dominated by grammatical stop-words (like "the", "of" and "in" etc.) These are essential for structure and removing them would lead to sort of loose translations.
- Hindi and Bengali: These both have prepositions, auxiliary verbs, conjunctions etc. as the most common tokens, again, removing them would be detrimental to the translations.

Hence, I didn't prune any part of the vocabulary. The final vocabulary sizes were around 57k, 75k and 105k for English, Hindi and Bengali respectively. The significantly larger Bengali vocabulary (105k) is in line with the much larger issue that I talked about above. This paper provides more detail: [3]

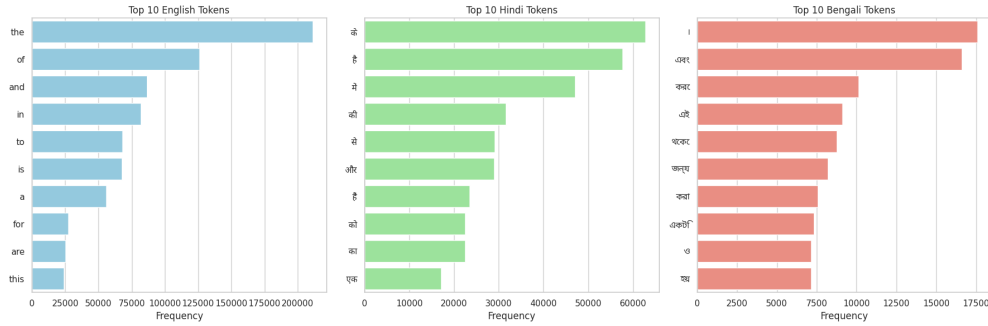


Figure 3: Histogram of most frequent tokens in the corpus

3.5 Final Pre-processing Decisions

Due to space constraints, I was not exactly able to explain in detail the inspirations I got from research papers in the above 3 subsections, but I have cited them.

Now, due to all the findings from the previous section, I determined that `nltk.word_tokenize` was probably sufficient for tokenization. I did experiment a bit with BPE (the one implemented in assignment 2) later, but didn't submit it as I was running into some issues. Hence, all my submissions just use `nltk` for tokenization. The final pre-processing pipeline was: Converting all text to lower-case → Removing punctuation and digits → Tokenization and cleaning using `nltk.word_tokenize` → Building vocabularies for both source and target languages including SOS, EOS and PAD tokens.

4 Model Description

This section will highlight the journey and an overall view of all the 25 train submissions made. It was a systematic approach that basically replicated the order of invention of these models, so I started with basic RNN based models and progressively increased the complexity of the models. (Note: chrF++ score is being referred to whenever I talk about "score/accuracy" in the remainder of the document.)

4.1 First Few Models: Baseline Seq2Seq Models

Submissions 1-5

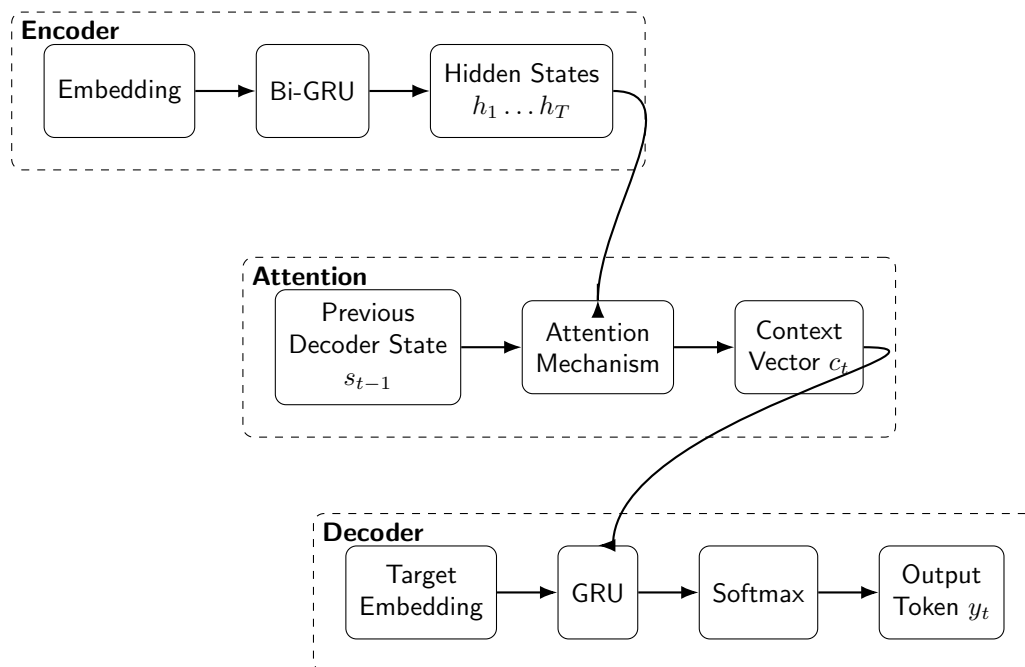
These submissions were essential in establishing baseline performances with simple models being implemented like vanilla RNN, LSTM and GRU. Obviously, single layer models were not suited for this as they couldn't take long term dependencies into account. I then moved on to Bi-LSTM, and Bi-GRU which gave about the same kind of performance.. out of this cluster of submissions, the best score was around 0.21, of the Bi-LSTM model.

4.2 Adding Attention to These Baselines

Submissions 6-12

Since there was a bottleneck of a fixed-sized context vector, it was only logical to include attention mechanisms next. So, I trained a Bi-GRU and Bi-LSTM with two types of attention mechanism (both of their papers are cited below), namely, **Bahdanau** [4] and **Luong** [5] attention. These models were some of the slowest to train and inference, but performed pretty well over the baseline, with a score of around 0.32. Stacked LSTMs and Stacked Bi-LSTMs were also tried out among these models, but didn't really perform that well compared to attention.

The graph shown below depicts the encoder-decoder architecture for these models:



4.3 Moving on to Transformers

Submissions 13-16

We needed the parallelization and self-attention features that a transformer has, hence, a generic transformer model was trained. A standard 3-layer transformer was implemented and trained for about 30 epochs, and even the vanilla transformer severely outperformed the best Seq-2-Seq based models, getting a score of around 0.35

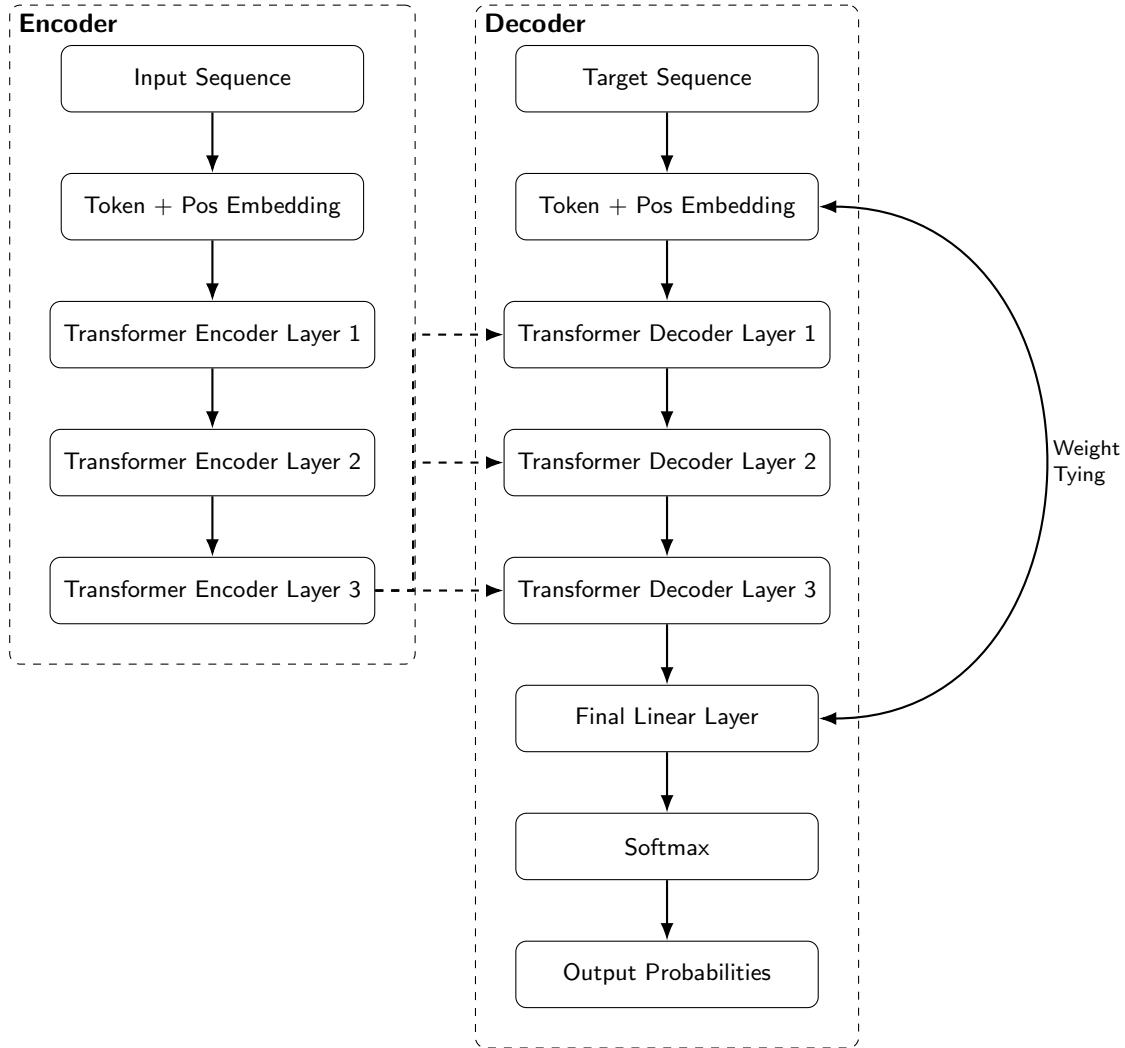
4.4 Optimizing the Vanilla Transformer

Submissions 17-25

This model was the final submission. I took inspiration from some very famous research papers and added two key differences in the existing transformer architecture:

1. **Noam Optimizer:** Training with the standard Adam optimizer was unstable. After going through the details for training in Vaswani et al. [6], I implemented the **custom learning rate schedule**, which significantly sped up and stabilized training.
2. **Weight Trying:** I read the paper by Press & Wolf [7] and some other articles online led me to the practice of "weight trying", which meant sharing weights between the decoder's embedding layer and final linear projection layer. This was effectively a regularization technique that led to the single largest performance jump. The final submission had a score of 0.41 (in around 30 epochs, but I think I could've pushed it further to get closer to convergence.)

The graph below will depict the best performing model's architecture.



Summary of the architecture:

- Standard encoder-decoder transformer with $N = 3$ layers, $d_{model} = 256$ (embedding dim) and feed forward layer dimension, $d_{FF} = 512$ and 8 attention heads.
- **Custom LR schedule (Noam):** This is implemented directly from the paper cited above, this schedule increases the learning rate linearly for the first 4000 warmup steps and then follows an inverse square root decay (ie, it decays the LR proportionally to the inverse sqrt of the current step.)
- **Weight Tying:** As shown in the diagram, the weights between decoder embedding and decoder FC layer are shared, ie, `self.decoder.fc_out.weight = self.decoder.tok.embedding.weight`. This reduces the total parameters of the model by millions and is a really good regularizer, because it ensures the embedding space and the logit space (or the final output space) are semantically similar.

4.5 Best Model Inference

The model training made it minimize the cross-entropy loss function (ignoring PAD tokens). For inference, **Greedy Decoding** was employed, Beam Search could've been implemented but there were computational constraints and greedy proved to give a pretty good score and position on the leaderboard.

5 Experiments

5.1 Data Pre-Processing

As described in section 3, the pre-processing pipeline (implemented in `preprocess_sentence` function) first normalises the text (by lowercasing, removing punctuation/digits) and then tokenizes it using `nlTK.word_tokenize`. These tokenized sentences are then converted into vectors using the `Vocab` object implemented for all 3 languages. Then all sequences are truncated or padded to length 25 (and SOS/EOS/PAD tokens are added according to this).

5.2 Training Procedure

The following things were implemented:

- The NoamOpt wrapper on top of `torch.optim.Adam` and custom Learning Rate schedule with 4000 warmup steps and model embedding dimension as 256.
- **Epochs:** The final model was trained for 21 epochs.
- **Training Time:** (on the kaggle GPU) was
 - Bengali (68k examples): approx 1 min 22 secs per epoch
 - Hindi (80k examples): approx 1 min 21 secs per epoch
- **Loss:** The training loss for the Bengali model decreased from 8.99 to 4.12 and for Hindi it decreased from 7.67 to 2.60 over the course of 21 epochs. (Cross-Entropy loss)

5.2.1 Hyperparameter Details

This is a summary table of the hyperparameters for the final model.

Hyperparameter	Value	Rationale
D_MODEL	256	Tradeoff b/w model capacity and compute cost
N_HEADS	8	D_MODEL must be divisible by N_HEADS.
N_LAYERS	3	Original paper had 6 but used 3 for faster train
D_FF	512	$2 \times D_MODEL$, by convention.
DROPOUT	0.1	Standard dropout rate (didn't change this).
BATCH_SIZE	64	Largest size that fits easily in GPU memory.
NUM_EPOCHS	21	Probably could have increased this
WARMUP_STEPS	4000	Value from the <i>Attention is All You Need</i> paper.
MAX_LENGTH	25	Refer to section 3 for this
CLIP	1.0	clipping to prevent exploding gradients.

6 Results

This table highlights the submission range according to the information in section 4, with the best model architecture in that category and the best chrF++ score.

Note: Explanation of these results has already been done earlier in this report.

Submission	Best Architecture	Best chrF++
Subs 1-5	Stacked Bi-LSTM (2-layer)	0.21
Subs 6-12	Bi-GRU + Bahdanau Attention [4]	0.27
Subs 13-16	Baseline Transformer (3-layer [6])	0.34
Subs 17-25	Optimized Transformer [6]	0.41

7 Error Analysis

1. Error analysis of your different models both on the dev set and the test set. What models worked in general and in what kind of setting? What was the reason for it?
 - Bi-GRU with Attention worked well as an initial seq2seq model, it was relatively simple to train and gave a solid baseline.
 - But, the optimized transformer version was superior in all settings, since attention mechanism was essential in capturing long term dependencies, and optimizations like weight trying and Noam really pushed its performance.
2. Error analysis includes analysis of the model, you can use visualizations for this. For e.g., if you used attention mechanism you can use, attention heat maps, etc.
 - Unfortunately, the `MultiHeadAttention` module returns `attention_weights` but I didn't get time to implement and look at attention heat maps. This would be a nice area to work on in the future though since it would help me look at exactly what the model is looking at and giving priority to during translation.
3. Analyze why models are not perfect? E.g., what kind of mistakes are made by the best model, how could these be overcome? The best model that I could come up with had a chrF++ score of 0.41, which was far from perfect and could be optimized by 3 main ways:
 - Changing the inference strategy from greedy decoding to beam search, since it would explore a much wider range of translations.
 - Changing the tokenization method from `nltk.word_tokenize` to some sub-word tokenization algorithm like BPE or SentencePiece. This is because it can handle OOV words and learn morphological nuances that are essential to indic languages.
 - Increasing model size: this is dependent heavily on compute access but in most of the papers I read, the transformers had 6 layers instead of 3 and had much higher dimensions. Obviously that would lead to much better translations even though it comes at a huge computational cost.
4. Any interesting insights!
 - Something interesting I observed was that the model performed slightly better on Hindi than Bengali, despite having a significantly larger Bengali vocabulary. This suggests that Bengali has a more nuanced morphology and structure that makes it hard for the model to translate compared to hindi.

8 Conclusion

The project involved a successful implementation of a complete Neural Machine Translation pipeline, and it translated English into Hindi and Bengali.

- **Key Findings:** Seq2Seq models like RNN-based models and transformer models were implemented in PyTorch and compared for this task, and the clear superiority of transformer architectures in this use-case was established. Also, clear improvements over vanilla transformers were demonstrated through the practices of weight trying and custom learning rate scheduling.
- **Future Directions:** I think there are a lot of directions one could take to further improve this model, but the three main suggestions I would have are:
 1. Implementing a beam search decoder in place of greedy to improve inference.
 2. Replace NLTK tokenization with a sub-word tokenizer to better handle rare words and morphologically rich languages
 3. Train a more powerful and larger model with twice the number of layers and embedding dimensions, so that model capacity is increased.

References

- [1] P. Koehn and R. Knowles, “Six challenges for neural machine translation,” in *Proceedings of the First Workshop on Neural Machine Translation*, (Vancouver, Canada), pp. 28–39, 2017.
- [2] P. Koehn, “Europarl: A parallel corpus for statistical machine translation,” in *Proceedings of the MT Summit X*, (Phuket, Thailand), pp. 79–86, 2005.
- [3] R. Kumar, S. Sanyal, and P. Talukdar, “A survey of neural machine translation for indian languages,” in *Proceedings of the 13th International Conference on Natural Language Processing (ICON)*, (Patiala, India), 2018.
- [4] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *arXiv preprint arXiv:1409.0473*, 2014.
- [5] M.-T. Luong, H. Pham, and C. D. Manning, “Effective approaches to attention-based neural machine translation,” *arXiv preprint arXiv:1508.04025*, 2015.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pp. 5998–6008, 2017.
- [7] O. Press and L. Wolf, “Using the output embedding to improve language models,” in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2017)*, (Valencia, Spain), pp. 175–180, April 2017.